

Day 27: セキュリティ対策を学ぼう

今日のゴール

Webアプリケーションの基本的なセキュリティ対策を学びます。パスワード保護、XSS対策、CSRF対策などを理解します。

【スクリーンショット: セキュリティチェックリスト】

なぜこれを学ぶのか?

セキュリティは信頼の基盤です。セキュリティは家の鍵のようなもの。鍵をかけずに外出すると、泥棒に入られる危険があります。同様に、Webアプリもセキュリティ対策をしないと、ハッカーに攻撃される危険があります。

学習ステップ一覧

ステップ	学習内容	所要時間
Step 1	パスワード保護の確認	15分
Step 2	XSS対策の理解	15分
Step 3	CSRF対策の確認	15分
Step 4	環境変数の管理	10分

合計時間: 約55分

Step 1: パスワード保護の確認 (15分)

パスワードハッシュ化:

```
// filepath: src/server/api/routers/auth.ts
import bcrypt from 'bcryptjs';

export const authRouter = createTRPCRouter({
  register: publicProcedure
    .input(z.object({
      email: z.string().email(),
      password: z.string().min(8),
      name: z.string(),
    }))
    .mutation(async ({ ctx, input }) => {
      // パスワードをハッシュ化
      const hashedPassword = await bcrypt.hash(input.password, 10);

      const user = await ctx.db.user.create({
        data: {
          email: input.email,
          name: input.name,
          password: hashedPassword, // ハッシュ化したものを保存
        },
      });

      return user;
    }),
});
```

重要なポイント:

- × 平文パスワードをDBに保存しない
- bcryptでハッシュ化して保存
- ソルト(salt)を自動生成 (bcryptが自動でしてくれる)

確認ポイント: データベースにハッシュ化されたパスワードが保存される

Step 2: XSS対策の理解 (15分)

XSS (Cross-Site Scripting) とは:

悪意のあるJavaScriptコードを注入される攻撃。

```
// × 危険な例 ( 使わないこと )
export default function DangerousComponent({ html }: { ht
  return <div dangerouslySetInnerHTML={{ __html: html }}
}

// 安全な例 ( Reactが自動でエスケープ )
export default function SafeComponent({ text }: { text: s
  return <div>{text}</div>;
}
```

Reactのデフォルト保護:

```
const userInput = '<script>alert("XSS")</script>';

// Reactは自動的にエスケープする
return <Typography>{userInput}</Typography>;
// 結果: &lt;script&gt;alert("XSS")&lt;/script&gt; と表示
```

入力バリデーション:

```
// filepath: src/server/api/routers/task.ts
export const taskRouter = createTRPCRouter({
  create: protectedProcedure
    .input(z.object({
      title: z.string()
        .min(1, 'タイトルは必須です')
        .max(255, 'タイトルは255文字以内です')
        .trim(), // 前後の空白を削除
      description: z.string()
        .max(5000, '説明は5000文字以内です')
        .optional(),
    }))
    .mutation(async ({ ctx, input }) => {
      // zodが自動的にバリデーション
    }),
});
```

確認ポイント: 悪意のあるコードが実行されない

Step 3: CSRF対策の確認（15分）

CSRF（Cross-Site Request Forgery）とは:

他のサイトから勝手にリクエストを送られる攻撃。

NextAuthの保護:

```
// filepath: src/server/auth.ts
// NextAuthは自動的にCSRF対策を行う
export const authOptions: NextAuthOptions = {
  adapter: PrismaAdapter(db),
  session: {
    strategy: 'jwt', // JWTトークンでセッション管理
  },
  // NextAuthが自動でCSRFトークンを発行・検証
};
```

tRPCの保護:

```
// filepath: src/server/api/trpc.ts
const createInnerTRPCContext = async (opts: CreateContextOptions) => {
  const session = await getServerAuthSession(opts);

  return {
    session,
    db,
  };
};

// protectedProcedure を使うと自動的に認証チェック
export const protectedProcedure = t.procedure.use(({ ctx, next } => {
  if (!ctx.session || !ctx.session.user) {
    throw new TRPCError({ code: 'UNAUTHORIZED' });
  }
  return next({
    ctx: {
      session: { ...ctx.session, user: ctx.session.user },
    },
  });
}));
```

確認ポイント: 認証されていないリクエストが拒否される

Step 4: 環境変数の管理 (10分)

機密情報の保護:

```
# filepath: .env
# × これらをGitにコミットしない
DATABASE_URL="postgresql://user:password@localhost:5432/d
NEXTAUTH_SECRET="super-secret-key-12345"

# .gitignore に .env を追加済み
```

```
# filepath: .env.example
# これはGitにコミットしてOK (値は入れない)
DATABASE_URL="postgresql://user:password@host:5432/databa
NEXTAUTH_SECRET="your-secret-key-here"
NEXTAUTH_URL="http://localhost:3000"
```

環境変数の検証:

```
// filepath: src/env.js
import { z } from 'zod';

const envSchema = z.object({
  DATABASE_URL: z.string().url(),
  NEXTAUTH_SECRET: z.string().min(32),
  NEXTAUTH_URL: z.string().url(),
});

// ビルド時にチェック
export const env = envSchema.parse(process.env);
```

確認ポイント: .envファイルがGitにコミットされていない

学んだこと

- パスワードハッシュ化: bcryptで安全に保存
- XSS対策: Reactの自動エスケープと入力バリデーション
- CSRF対策: NextAuthとtRPCの自動保護
- 環境変数: 機密情報を.envで管理、Gitにコミットしない
- zod: 入力バリデーションでデータの安全性を確保

今日のまとめ

- ☐ パスワードがハッシュ化されていることを確認した
- ☐ XSS対策を理解した
- ☐ CSRF対策を確認した
- ☐ 環境変数を適切に管理できた

⚠ セキュリティチェックリスト

項目	確認方法
パスワードハッシュ化	DBに平文パスワードがないか確認
XSS対策	<code>dangerouslySetInnerHTML</code> を使っていないか
CSRF対策	<code>protectedProcedure</code> を使っているか
環境変数	<code>.env</code> が <code>.gitignore</code> に含まれているか
入力バリデーション	<code>zod</code> でバリデーションしているか
HTTPSの使用	本番環境では必ずHTTPSを使用

次回予告

Day 28では、テストの書き方を学びます。